VEXON DEVELOPER HANDBOOK · GUIDE 04

Database Guide - MongoDB Today, PostgreSQL Later

VEXON's real Mongoose models, queries and indexes, safe backups, and a plan (not a migration) for PostgreSQL

Written for	A student who must keep VEXON's data safe and may one day lead a migration to PostgreSQL
Project	VEXON / CampusConnect (repository: this folder's parent project)
Evidence	Generated from a direct inspection of the repository source code
Repository state	branch fix/csrf-legacy-cookie, commit af623fb, plus 126 uncommitted working-tree change(s)
Generated	16 October 2026, 21:11
Rebuild	python docs/developer-handbook/build_handbook.py

Accuracy rule: every statement about VEXON in this guide is based on files in the repository. Anything that could not be confirmed is labelled 'unverified', 'planned' or 'hypothetical'. No secrets, passwords or real student data are reproduced.

Contents

What VEXON uses today . . . 3
MongoDB in five minutes . . . 3
The models, grouped by module . . . 3
The relationship hub: User . . . 4
Create, read, update, delete: real examples . . . 4
Reading an aggregation . . . 5
Validation, indexes and integrity . . . 5
Validation . . . 5
Indexes that enforce rules . . . 6
TTL indexes: data that deletes itself . . . 6
How database errors appear in the backend . . . 7
Backup and restoration . . . 7
Inspecting records safely . . . 8
PostgreSQL: status and a safe migration plan . . . 8
Should VEXON migrate at all? . . . 8
Schema mapping, by example . . . 8
A safe sequence . . . 9
Practice exercises (no production data is modified) . . . 10
Source references . . . 11

What VEXON uses today

> **Evidence status**
> **MongoDB is the only database in VEXON** (implemented and verified: the server test suite runs against a real MongoDB). **PostgreSQL is not integrated.** No `pg`, Prisma, Sequelize, Knex or SQL file exists anywhere in the repository. Everything about PostgreSQL in this guide is a **plan for future work**, not a description of the system.

Item	Value (from the code)
Database engine	MongoDB. The README asks for Community Server 6.0+; CI uses the `mongo:7` image (`.github/workflows/ci.yml`)
Driver / ODM	Mongoose 8 (`server/package.json`)
Connection code	`server/src/config/db.js` (`connectDB`)
Which database	`server/src/config/env.js`: `DB_TARGET=local` uses `MONGO_URI` (default database `campusconnect` on `127.0.0.1:27017`); `DB_TARGET=atlas` uses `MONGO_URI_ATLAS`
Test database	`cmp_test`. `server/tests/helpers.js` refuses any database whose name does not end in `_test`, and drops it before and after each test file
Models	59 files in `server/src/models/`, one collection each
Production	MongoDB Atlas, for the Render deployment (`render.yaml`: `MONGO_URI` is set in the dashboard, never in the file)

MongoDB in five minutes

- A **database** (`campusconnect`) holds **collections** (`users`, `gatepasses`, ...). Collections hold **documents**: JSON-like objects with an `_id` (an ObjectId).
- Documents in one collection can differ in shape. **Mongoose adds the discipline**: a schema says which fields exist, their types, required fields, allowed values and indexes.
- **Relationships** are stored as ObjectIds that point to other documents (`ref: 'User'`). They are *not* enforced by the database: deleting a user does not delete its gate passes. Application code must handle that.
- Some data is **embedded** instead of referenced. Examples: `registrations` inside an `Event`, review stages inside a `GatePass`, `history` inside an `AttendanceRecord`.

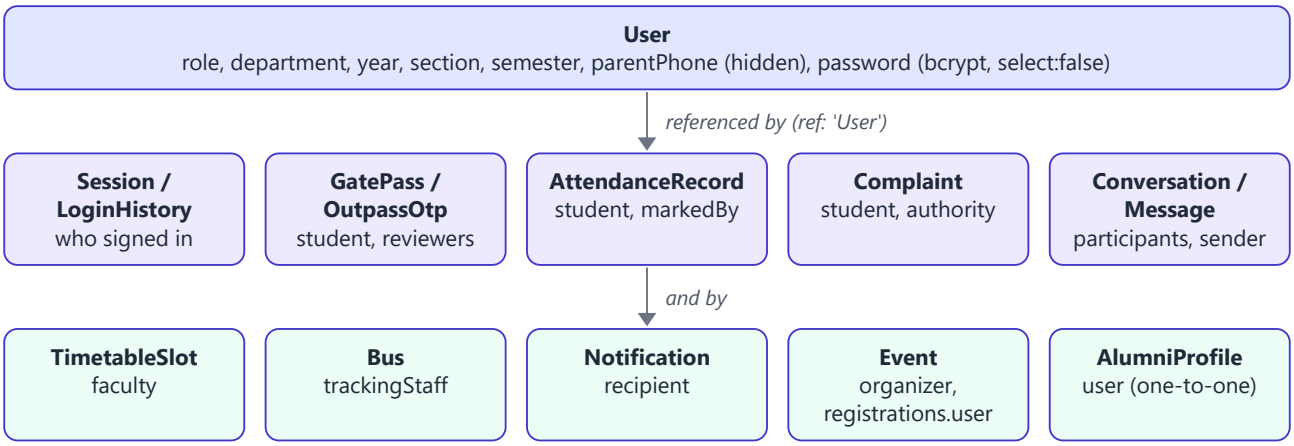
The models, grouped by module

Module	Models (files in `server/src/models/`)
Accounts and security	`User`, `Session`, `LoginHistory`, `LoginThrottle`, `SecurityLog`, `Activity`, `DeviceToken`
Community	`Announcement`, `Club`, `Event`, `Discussion`, `Report`, `Notification`, `Achievement`
Academics	`Subject`, `TimetableSlot`, `Exam`, `Holiday`, `StudyMaterial`, `StudyMaterialChunk`, `PeriodCover`
Attendance	`AttendanceSession`, `AttendanceRecord`, `AttendanceCorrectionRequest`, `FacultyAttendance`, `FacultyEarlyExit`, `ODLeaveRequest`
Gate pass	`GatePass`, `GatePassBindGroup`, `OutpassOtp`, `ReturnLocationCheck`
Campus	`LostFoundItem`, `Complaint`, `ComplaintHistory`, `SOSAlert`
Chat	`Conversation`, `Message`
Transport	`Bus`, `BusRoute`, `BusTrip`, `BusLocation`, `TransportAssignment`

Module	Models (files in `server/src/models/`)
AI	`AIConversation`, `AIUsageLog`
Alumni	`AlumniProfile`, `AlumniJob`, `JobApplication`, `AlumniEvent`, `AlumniEventRsvp`, `AlumniImportBatch`, `AlumniInvite`, `Chapter`, `ChapterMember`, `ChapterPost`, `ChapterComment`, `MentorshipRequest`, `MentorshipSession`, `MentorshipSlot`
Files	`StoredFile` (uploads stored in the database in production when Cloudinary is not set)

The relationship hub: User

Almost every model points to `User`:



Read `server/src/models/User.js` in full once. Notice these things:

- `password: { ..., select: false }`. The hash is never loaded unless code asks with `.select('+password')`, as `authController.login` does.
- `role: { enum: ROLES, default: 'student', index: true }`. Only the 15 known roles are accepted.
- Indexes: `class_roster` (role + department + year + section + semester), unique `employeeId` and unique `rollNo` (both *partial*, so an empty value never collides), and a weighted **text index** for search.
- `pre('validate')` hooks enforce business rules for every write path (the Class In-Charge year/semester pair; title vs gender).
- `pre('save')` hashes the password with bcrypt (12 rounds) whenever it changes.
- `toJSON` deletes secrets (password, tokenVersion, lock fields, reset hashes, push tokens) before anything is sent to a client.

Create, read, update, delete: real examples

Operation	Real example	File
Create	`await SOSAlert.create(data)`	`server/src/controllers/sosController.js`
Create (retry on duplicate code)	Loop around `Complaint.create({... complaintCode: generateComplaintCode()})`, retrying on error `11000`	`server/src/controllers/complaintController.js`
Read one	`User.findById(payload.sub)`	`server/src/middleware/auth.js`
Read many, trimmed	`Session.findById(sid).select('user revokedAt expiresAt lastUsedAt client').lean()`	`server/src/middleware/auth.js`

Operation	Real example	File
Count	`SOSAlert.countDocuments({ user, createdAt: { $gte: ... } })`	`sosController.js`
Distinct ids	`User.find({ role: { $in: roles }, isActive: true }).distinct('_id')`	`server/src/utils/notify.js` (`notifyRoles`)
Update one field	`Session.updateOne({ _id }, { $set: { lastUsedAt: new Date() } })`	`middleware/auth.js`
Bulk insert	`Notification.insertMany(docs, { ordered: false })`	`utils/notify.js`
Full-text search	`Model.find({ ...filter, $text: { $search: q } }, { score: { $meta: 'textScore' } })`	`server/src/controllers/searchController.js`
Aggregation	`AttendanceRecord.aggregate([{ $match }, { $group }, { $addFields }, { $sort }, { $project }])`	`server/src/controllers/analyticsController.js`

Query operators you will meet: `$in`, `$gte` / `$lte` (date ranges), `$set`, `$addToSet`, `$exists`, `$or`, and `$text`. Aggregation stages: `$match` (filter), `$group` (totals per key), `$lookup` (join another collection), `$project` (shape the output) and `$sort`.

Reading an aggregation

```js
// server/src/controllers/analyticsController.js
async function attendanceTotals(match) {
  const [row] = await AttendanceRecord.aggregate([{ $match: match }, { $group: { _id: null, total: {
    $sum: 1 }, present: presentExpr } }]);
  const total = row?.total || 0;
  const present = row?.present || 0;
  // Σ present ÷ Σ conducted — never an average of per-subject percentages.
  return { totalPeriods: total, presentPeriods: present, absentPeriods: total - present, percentage:
    percentage(present, total) };
}
```

- `$match` keeps only the records in scope (a student, a class, a date range).
- `$group` with `_id: null` folds them into **one** row. `total` counts every record, and `present` (`presentExpr`) counts the present ones.
- `const [row] = ...` takes the first (only) result. If no records exist there is no row, so `row?.total || 0` returns 0 instead of crashing.
- The comment states a business rule: overall percentage is total present divided by total conducted, **not** an average of subject percentages.

Validation, indexes and integrity

Validation

Mongoose validates on `create()` and `save()`: `required`, `enum`, `min`/`max`, `maxlength`, `match`, and custom `validate` functions and `pre('validate')` hooks. Note that `updateOne`/`updateMany` **skip** most validation unless you pass `{ runValidators: true }`.

Indexes that enforce rules

Some indexes do more than speed things up. They **prevent bad data**:

Index	Model	Rule it enforces
`unique_attendance` on student + subject + date + period	`AttendanceRecord.js`	One attendance mark per student per period
`unique_active_slot` (partial: `isActive: true`)	`TimetableSlot.js`	No double-booked class period
`email` unique	`User.js`	One account per email
`unique_employee_id`, `unique_roll_no` (partial)	`User.js`	Ids unique when present
(user, eventKey) unique	`Notification.js`	The same event never notifies a user twice (`notifyUsers` relies on it)

TTL indexes: data that deletes itself

Collection	Kept for	Defined in
`Activity`	180 days	`models/Activity.js`
`AIUsageLog`	180 days	`models/AIUsageLog.js`
`SecurityLog`	365 days	`models/SecurityLog.js`
`Notification`, `LoginHistory`	90 days	`models/Notification.js`, `models/LoginHistory.js`
`ReturnLocationCheck`	90 days	`models/ReturnLocationCheck.js`
`OutpassOtp`	30 days	`models/OutpassOtp.js`
`Session`	30 days after expiry	`models/Session.js`
`BusLocation`	`TRANSPORT_LOCATION_RETENTION_DAYS` (default 7, max 365)	`models/BusLocation.js`
`LoginThrottle`	until its `expiresAt`	`models/LoginThrottle.js`

`connectDB()` calls `syncIndexes()` at start-up, so new indexes are created automatically. `syncIndexes` **also drops indexes that are no longer declared.** Removing an index line from a model removes the index from the database on the next start.

> **Warning**
> Adding a **unique** index to a collection that already holds duplicates makes index creation fail at start-up. Before adding one, check for duplicates with an aggregation such as `[{ $group: { _id: { a: '$a', b: '$b' }, n: { $sum: 1 } } }, { $match: { n: { $gt: 1 } } }]`.

How database errors appear in the backend

What happened	What you see	Handled in
MongoDB not running / wrong URI	`[db] Could not connect to MongoDB (local), attempt 1/5: connect ECONNREFUSED 127.0.0.1:27017`, then exit after 5 tries	`config/db.js`
Atlas IP not allowed	Same, plus the hint about the Atlas Network Access list	`config/db.js`
Invalid ObjectId in a query	400 `Invalid _id` (`CastError`)	`middleware/error.js`
Schema validation failed	422 with `errors: [{ field, message }]`	`middleware/error.js`
Duplicate unique key	409 `That email is already in use` (code `11000`)	`middleware/error.js`
Transaction on a standalone server	Not an error: `withTransaction` detects it and runs without one	`utils/transaction.js`

Backup and restoration

> **Evidence status**
> **No backup script exists in the repository.** The procedures below are standard MongoDB practice, adapted to VEXON's setup. They are not automated. The separate `docs/VEXON_VPS_Setup_Guide_Ubuntu24.pdf` describes a planned VPS backup design (replica plus dumps); it states its commands "were not run".

Tools: `mongodump` and `mongorestore` come in the *MongoDB Database Tools* package, which is installed separately from the server. On Atlas, the paid tiers also offer automated snapshots. Check what your plan includes.

Local development database:

```bash
# Back up (creates a folder of BSON files). Keep backups OUTSIDE the repository.
mongodump --uri "mongodb://127.0.0.1:27017/campusconnect" --out "D:/vexon-backups/2026-10-16"

# Restore into a SEPARATE database first, to test the backup without touching the original
mongorestore --uri "mongodb://127.0.0.1:27017" --nsFrom "campusconnect.*" --nsTo
    "campusconnect_restorecheck.*" "D:/vexon-backups/2026-10-16"
```

Atlas (production): use the connection string from your password manager or the Render dashboard. **Never paste it into a document, chat or commit.**

```bash
mongodump --uri "$MONGO_URI" --out ./backup-YYYY-MM-DD      # run from a trusted machine
```

A backup counts only once you have **restored it successfully**. Monthly, restore to a scratch database and compare document counts per collection (`db.users.countDocuments()` and so on).

> **Warning**
> Backups contain personal data: names, emails, parents' phone numbers, attendance and chat. Encrypt them, keep them off shared drives, and never commit them. `.gitignore` already ignores `/*.pdf` and `.env*`, but **not** dump folders, so store them outside the repository.

Uploaded files are part of the data too. They live in Cloudinary, in the `StoredFile` collection (production without Cloudinary), or in `server/uploads/` (development). A complete backup covers all three.

Inspecting records safely

Use **mongosh** or **MongoDB Compass** against a *development* or *restored copy*, not production, unless you must.

```
js
// mongosh, connected to the dev database
use campusconnect
db.users.countDocuments({ role: 'student' })
db.users.find({ role: 'hod' }, { name: 1, department: 1, _id: 0 }).limit(5)   // project only harmless
    fields
db.gatepasses.aggregate([{ $group: { _id: '$status', n: { $sum: 1 } } }])      // counts, no personal data
```

Rules for yourself:

- Prefer **counts and group-bys** over dumping whole documents.
- Use a **projection** (`{ name: 1, _id: 0 }`) so phone numbers, parent numbers and tokens are never printed.
- Never paste real records into an AI chat, an issue or a screenshot. Copy a document and replace the values with fake ones first.
- Chat message bodies are **encrypted at rest** (AES-256-GCM, `server/src/utils/cipher.js`, key from `CHAT_ENCRYPTION_KEY`). Seeing ciphertext in the database is normal. Losing or changing the key makes old messages unreadable.

PostgreSQL: status and a safe migration plan

> **Evidence status**
> Everything in this section is **planned, not implemented**. No migration has been run. This handbook did not create one, and none should be run without a tested plan and a fresh, verified backup.

Should VEXON migrate at all?

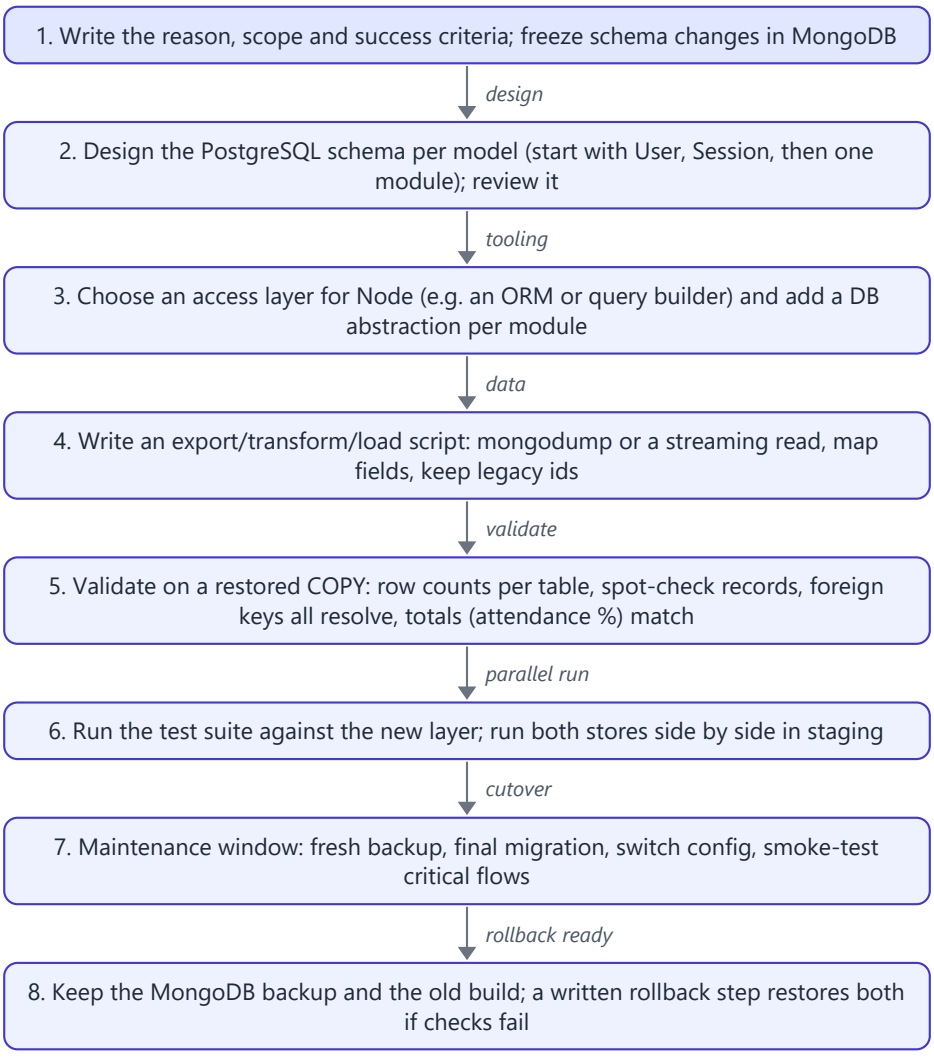
MongoDB works for VEXON today, and about 300 routes and 59 models depend on Mongoose. A migration is a large project, justified only by a concrete need. Examples: reporting needs heavy SQL joins, the college mandates PostgreSQL, or relational integrity (foreign keys) is wanted for attendance and gate-pass data. Write down the reason before starting.

Schema mapping, by example

Mongo concept	PostgreSQL equivalent	VEXON example
Collection	Table	`users`
`_id` ObjectId	Primary key. Keep the old id in a `legacy_id CHAR(24) UNIQUE` column, or use it as the key	Preserves links in URLs and notifications (`/gate-pass/<id>`)
`ref: 'User'`	`user_id ... REFERENCES users(id)`	`gate_passes.student_id`
Embedded array of objects	Child table	`Event.registrations` becomes `event_registrations(event_id, user_id, status, ...)`
Embedded single object	Columns, or `JSONB`	`GatePass` review stages: `faculty_review_by`, `faculty_review_at`, ...

Mongo concept	PostgreSQL equivalent	VEXON example
Array of strings	`TEXT[]`, or a join table	`User.skills`, `User.interests`
`enum`	`CHECK` constraint or a PostgreSQL `ENUM` type	`role IN ('student', ...)`
Unique / partial unique index	`UNIQUE` / partial `UNIQUE ... WHERE`	`unique_active_slot WHERE is_active`
TTL index	A scheduled `DELETE ... WHERE created_at < now() - interval '90 days'`	Notifications, logs
Text index	`tsvector` + GIN index	Global search
Aggregation pipeline	SQL `GROUP BY`, `JOIN`, window functions	Analytics

A safe sequence



Identifier preservation matters because ids appear in notification links, QR flows, socket rooms (`chat:<id>`, `bus:<id>`) and the mobile apps' cached routes. Losing them breaks history and links.

Validation checklist: counts per collection equal counts per table; every foreign key resolves; no `NULL` in required columns; unique constraints hold; computed results (attendance percentages, gate-pass counts, complaint stages) match between the old and new systems for the same inputs.

Rollback: keep the old application version deployable, keep the pre-migration dump, and decide *in advance* the conditions under which you roll back.

Practice exercises (no production data is modified)

Exercise

1. In `server/src/models/AttendanceRecord.js`, which index prevents marking the same student twice for one period? **Expected:** `unique_attendance` on `{ student, subject, date, period }`.

Exercise

2. In mongosh on your **dev** database, count gate passes per status with `$group`. **Expected:** a short list of statuses, each with a number. No names or phone numbers are printed.

Exercise

3. Why does `User.js` use a *partial* unique index for `rollNo`? **Expected:** staff have no roll number. A plain unique index would treat every missing value as equal and allow only one such user.

Exercise

4. Back up your dev database with `mongodump` to a folder outside the repository, then restore it into `campusconnect_restorecheck`. Compare `db.users.countDocuments()` in both. **Expected:** equal counts. Then drop the check database (`use campusconnect_restorecheck; db.dropDatabase()`). Double-check the name before you press Enter.

Exercise

5. Map `server/src/models/Complaint.js` to a PostgreSQL design on paper: tables, columns, keys. **Expected:** a `complaints` table, plus a `complaint_history` table referencing it, plus an attachments child table or a `JSONB` column.

Source references

server/src/config/env.js · server/src/config/db.js · server/src/models/User.js · server/src/models/AttendanceRecord.js · server/src/models/TimetableSlot.js · server/src/models/Notification.js · server/src/models/Session.js · server/src/models/Activity.js · server/src/models/SecurityLog.js · server/src/models/BusLocation.js · server/src/models/OutpassOtp.js · server/src/models/Complaint.js · server/src/middleware/error.js · server/src/utils/transaction.js · server/src/utils/cipher.js · server/src/utils/notify.js · server/src/controllers/analyticsController.js · server/src/controllers/searchController.js · server/tests/helpers.js · .github/workflows/ci.yml · render.yaml · docs/VEXON_VPS_Setup_Guide_Ubuntu24.pdf